TRACKER_AUTH_TESTING

Tracker Credentials Live Testing

Revision: 1 **Last modified:** 2026-06-14T00:00:00Z **Status:** active **Status summary:** Live credential-validation tests + challenge for the 5 supported trackers, wired into the integration suite and the challenge bank.

Table of contents

- Overview
- The five trackers
- How the live tests validate credentials
- Honest-SKIP vs FAIL rules
- Security: never read or log credential values
- How to run
- Where these are wired

Overview

Two artefacts live-validate that the merge service can authenticate against each supported tracker using the credentials configured in `.env`:

- `tests/integration/test_tracker_auth_live.py` — a parametrized pytest live integration test, one parameter per tracker.
- `challenges/scripts/tracker_auth_live_challenge.sh` — a bash challenge that drives the same end-to-end auth path.

Both exercise the **real running merge service** (port `7187`) and assert on its reported authentication state. They never inspect, print, or log any credential value (constitution §11.4.10).

The five trackers

Tracker	Type	Auth model
`rutracker`	private	Username/password login (cookies; CAPTCHA may be required periodically)
`kinozal`	private	Username/password (`KINOZAL_*`, falling back to `IPTORRENTS_*`)
`nnmclub`	private	Cookie-based auth (`NNMCLUB_COOKIES`)
`iptorrents`	private	Username/password (`IPTORRENTS_*`); freeleech-only downloads
`rutor`	public	**No authentication** — searches and downloads anonymously

For the four private trackers the test asserts that valid credentials produce an authenticated session. For `rutor` there is no login endpoint; the test asserts only that it is reachable and needs no auth (a positive auth assertion is intentionally not made).

How the live tests validate credentials

The live test/challenge validates each credential by driving a REAL search through the merge service and reading the per-tracker authentication state the service itself reports during that search:

1. Confirm the merge service is up (`GET /health`).
2. Start a real search — `POST /api/v1/search` with a public query (`{"query": "ubuntu", "limit": 50}`) — then poll `GET /api/v1/search/{search_id}` until the search reaches a terminal state (`completed` / `no_results`).
3. The response carries a `tracker_stats` list — one entry per tracker — each shaped `{name, status, authenticated, results_count, error}`. For each **private** tracker (rutracker, kinozal, nnmclub, iptorrents) assert `authenticated` is `true` AND `status` is `success` or `empty` — proof the stored credentials logged in against the real tracker. For **rutor** (public, no login) assert `status == "success"` and `results_count > 0`.

The assertion oracle is the **`authenticated` flag the merge service reports for the tracker after attempting a real login during the search** — a user-observable outcome of the real auth path — never the mere absence of an error and never any credential string (anti-bluff,

constitution §11.4 / §11.4.10). No `/api/v1/auth/*` status endpoint is consulted; the search response's `tracker_stats` is the single source of truth.

Honest-SKIP vs FAIL rules

Per constitution §11.4.3 (topology-appropriate dispatch) and §11.4.69 (no fail-open SKIP), the verdicts are:

- **SKIP** — the merge service is **down** / **unreachable**: the topology required to validate is absent, so the test SKIPs with a reason rather than failing.
- **SKIP** — a **CAPTCHA or transient** blocks login (e.g. RuTracker cookies expired and a CAPTCHA is now required): this is an operator-attended/transient condition, not a product defect, so it SKIPs with a reason.
- **FAIL** — the service is up, no captcha/transient is in play, and the tracker reports `authenticated=false`: the configured credentials are genuinely **bad** (or the auth path is broken). This is a real failure.

A SKIP is never silently counted as a PASS, and a missing/empty `authenticated` flag is treated as a failure to prove auth, not as proof.

Security: never read or log credential values

- The tests assert **only** on the service-reported `tracker_stats[].authenticated` boolean (plus `status` / `results_count`). They never read `RUTRACKER_PASSWORD`, `KINOZAL_PASSWORD`, `NNMCLUB_COOKIES`, `IPTORRENTS_PASSWORD`, or any other secret value, and never echo container env values.
- `.env` is never committed (constitution §11.4.10; project `CLAUDE.md`).
- No credential value appears in test output, challenge output, or logs.

How to run

Prerequisite: the merge service must be running on port `7187` with valid credentials present in `.env` (otherwise the tests SKIP, see above).

```
# Pytest live integration test (auto-collected by ci.sh PHASE 4)
python3 -m pytest tests/integration/test_tracker_auth_live.py -v
        --import-mode=importlib
```

```
# The bash challenge (auto-discovered by the challenge aggregator)
bash challenges/scripts/tracker_auth_live_challenge.sh

# Or as part of the wider suites:
./ci.sh                                  # PHASE 4 collects tests/
        integration/
scripts/run-tests.sh live                # tests/integration +
        tests/e2e
bash challenges/scripts/run_all_challenges.sh
```

Where these are wired

- **Pytest collection** — `tests/integration/test_tracker_auth_live.py` is auto-collected by `ci.sh` PHASE 4 (`pytest .../tests/integration/`) and by `scripts/run-tests.sh live`. No collection change was needed (default `python_files = test_*.py`; no override in `pyproject.toml`).
- **Challenge aggregator** — `challenges/scripts/run_all_challenges.sh` globs `*_challenge.sh` in its own directory, so `tracker_auth_live_challenge.sh` is auto-discovered there. No explicit registration was needed.
- **HelixQA bank** — `BOBA-SRV-015` in `submodules/helixqa/banks/boba-services.yaml` (symlinked into `challenges/helixqa-banks/boba-services.yaml`) references both artefacts by path with an anti-bluff note that the `authenticated` flag is the assertion oracle and that no credential value is ever read.